TP7 — Collections

List, Set, Map, Iterator, Comparable, import CSV de films

Sommaire

1. Objectif du TP
2. Notions de cours
3. Code du TP (commenté)
4. Résumé express

1. Objectif du TP

Manipuler les principales **collections** de l'API Java : listes (`List`/`ArrayList`), ensembles (`Set`/`TreeSet`/`HashSet`), dictionnaires (`Map`/`HashMap`), les différents parcours (for, for-each, `Iterator`), le tri (`Collections.sort`, `Comparable`) et l'import d'un fichier CSV de films.

2. Notions de cours

Pourquoi des collections ?

Un tableau a une taille fixe et peu de méthodes. Les collections (`java.util`) sont **dynamiques** (elles grandissent toute seules) et offrent des méthodes prêtes à l'emploi : ajout, suppression, recherche, tri... Elles ne contiennent que des **objets** : pour des nombres on utilise les wrappers (`List<Integer>`).

Interfaces et implémentations

Interface	Caractéristiques	Implémentations courantes
`List<E>`	Ordonnée (indices), **doublons autorisés**	`ArrayList` (accès rapide par indice), `LinkedList`
`Set<E>`	**Pas de doublons**	`HashSet` (aucun ordre garanti, très rapide), `LinkedHashSet` (ordre d'insertion), `TreeSet` (**trié**)
`Map<K,V>`	Associations **clé** → **valeur**, clés uniques	`HashMap` (aucun ordre), `LinkedHashMap`, `TreeMap` (clés triées)

> **Astuce :** Bonne pratique : déclarer avec l'**interface** et instancier avec l'**implémentation** : `List<String> couleurs = new ArrayList<>();`. On peut ensuite changer d'implémentation sans toucher au reste du code. `<>` (*diamond*) laisse Java déduire le type générique.

Génériques

`List<String>` signifie « liste qui ne contient **que** des `String` ». Le compilateur vérifie les types (impossible d'ajouter un `Integer`) et il n'y a pas besoin de cast à la lecture.

Méthodes essentielles

List	Set	Map
`add(e)`, `add(i, e)` (insère à l'indice i)	`add(e)` (retourne false si déjà présent)	`put(k, v)` (remplace si k existe)
`get(i)`, `set(i, e)`, `remove(i)`	`contains(e)`, `remove(e)`	`get(k)` (null si absent), `getOrDefault(k, d)`

List	Set	Map
`size()`, `isEmpty()`, `contains(e)`, `indexOf(e)`	`size()`, `isEmpty()`	`containsKey(k)`, `keySet()`, `values()`, `entrySet()`
`Collections.sort(l)`, `Collections.reverse(l)`	`new TreeSet<>(collection)`	`computeIfAbsent(k, f)`

Trois façons de parcourir une liste

Parcours

```
// 1. for classique avec indice
for (int i = 0; i < liste.size(); i++) { System.out.println(liste.get(i)); }

// 2. for-each (le plus lisible) - fonctionne avec toute collection et les tableaux
for (String s : liste) { System.out.println(s); }

// 3. Iterator - permet de SUPPRIMER pendant le parcours (it.remove())
Iterator<String> it = liste.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("rouge")) it.remove();
}
// (supprimer avec liste.remove(...) dans un for-each -> ConcurrentModificationException)
```

Comparable et ordre naturel

- Pour trier des objets (`Collections.sort`, `TreeSet`, `TreeMap`), Java doit savoir les comparer : la classe implémente `Comparable<T>` et sa méthode `int compareTo(T autre)`.
- `compareTo` retourne un nombre **négatif** si `this < autre`, **0** si égaux, **positif** si `this > autre`.
- `String`, `Integer`, `Year`, `LocalDate`... implémentent déjà `Comparable` (ordre alphabétique / chronologique).
- **Piège** : un `TreeSet` considère comme **doublons** deux objets dont `compareTo` renvoie 0. Si on compare les films uniquement sur l'année, un seul film par année est conservé ! D'où la comparaison année **puis** titre (exercice 7).
- `HashSet`/`HashMap` utilisent `equals()` **et** `hashCode()` : il faut redéfinir les deux ensemble.

3. Code du TP (commenté)

Exercice 1 — Nuancier (List / ArrayList)

nuancier/Nuancier.java

```
package net.lecnam.ecue2.tp7.nuancier;

import java.util.ArrayList;
import java.util.List;

public class Nuancier {

    // Déclaré avec l'INTERFACE List, générique <String> ; non initialisé ici
    private List<String> couleurs;

    // Constructeur sans paramètre
    public Nuancier() {

        // Initialisation avec l'implémentation ArrayList (<> : type déduit)
        couleurs = new ArrayList<>();

        // add(e) ajoute en FIN de liste
        couleurs.add("noir");
        couleurs.add("rouge");
```

nuancier/Nuancier.java

```
        couleurs.add("vert");
        couleurs.add("marron");
        couleurs.add("blanc");

        // Une List accepte les doublons : "rouge" est présent 2 fois
        couleurs.add("rouge");

        // add(indice, e) INSÈRE à la position 0 et décale tous les autres éléments
        couleurs.add(0, "cyan");
        // Résultat : [cyan, noir, rouge, vert, marron, blanc, rouge]  (7 éléments)
    }

    // Getter (/!\ renvoie la liste interne : l'appelant peut la modifier, cf. TP5)
    public List<String> getCouleurs() {
        return couleurs;
    }
}
```

Execute.java — exercice1()

```
package net.lecnam.ecue2.tp7;

import java.util.List;
import java.util.Iterator;
import java.util.Collections;

import net.lecnam.ecue2.tp7.nuancier.Nuancier;   // classe d'un AUTRE package : import obligatoire

public class Execute {

    public static void main(String[] args) {
        exercice1();
        //exercice2();
        //exercice3();
        //exercice4();
        //exercice5();
        //exercice6();
    }

    private static void exercice1() {
        Nuancier nuancier = new Nuancier();
        List<String> couleurs = nuancier.getCouleurs();

        // size() : nombre d'éléments -> 7
        System.out.println("Nombre de couleurs : " + couleurs.size());

        // 1. Parcours avec un for classique (indices de 0 à size()-1)
        System.out.println("Parcours avec for :");
        for (int i = 0; i < couleurs.size(); i++) {
            System.out.println(couleurs.get(i));
        }
        System.out.println("----------------");

        // 2. Parcours avec for-each
        System.out.println("Parcours avec foreach :");
        for (String couleur : couleurs) {
            System.out.println(couleur);
        }
        System.out.println("----------------");

        // 3. Parcours avec un Iterator : hasNext() teste, next() avance et renvoie
        System.out.println("Parcours avec Iterator :");
        Iterator<String> iterator = couleurs.iterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }
        System.out.println("----------------");

        // get(0) : premier élément -> "cyan"
        System.out.println("Première couleur : " + couleurs.get(0));
```

Execute.java — exercice1()

```
        // Tri selon l'ordre naturel des String (alphabétique). Trie la liste EN PLACE.
        Collections.sort(couleurs);
        System.out.println("----------------");
        System.out.println("Liste triée :");
        for (String couleur : couleurs) {
            System.out.println(couleur);  // blanc, cyan, marron, noir, rouge, rouge, vert
        }
    }

    // exercice2() ... exercice6() : vides dans ton projet (solutions ci-dessous)
}
```

Exercices 2 et 3 — TreeSet et HashMap (non faits : solution proposée)

Ajouts dans Nuancier.java

```
import java.util.*;

public class Nuancier {
    private List<String> couleurs;
    private Map<Character, String> couleursMap;    // clé : 1re lettre, valeur : couleur

    public Nuancier() {
        // ... (ajouts des couleurs comme avant)
        initializeMap();      // appelée en DERNIER : la liste doit déjà être remplie
    }

    // Exercice 2 : un TreeSet construit à partir de la liste, en une ligne.
    // Un Set supprime les doublons ("rouge" n'apparaît plus qu'une fois)
    // et un TreeSet trie les éléments (ordre alphabétique).
    public Set<String> getCouleursTreeSet() {
        return new TreeSet<>(couleurs);
    }

    // Exercice 3 : association initiale -> couleur
    private void initializeMap() {
        couleursMap = new HashMap<>();
        for (String couleur : getCouleursTreeSet()) {
            // charAt(0) : 1re lettre. put() REMPLACE la valeur si la clé existe déjà :
            // "marron" puis... aucune autre couleur en m, mais si on avait "magenta",
            // une seule des deux serait conservée (clés uniques).
            couleursMap.put(couleur.charAt(0), couleur);
        }
    }

    public String getCouleurFromCode(char initiale) {
        return couleursMap.get(initiale);    // null si aucune couleur ne commence par cette lettre
    }
}

// Dans Execute :
private static void exercice2() {
    // Affiche [blanc, cyan, marron, noir, rouge, vert] :
    // le doublon "rouge" a disparu (Set) et l'ensemble est trié (Tree)
    System.out.println(new Nuancier().getCouleursTreeSet());
}
private static void exercice3() {
    System.out.println(new Nuancier().getCouleurFromCode('r'));   // rouge
}
```

Exercice 4 — Classe Film (Comparable)

film/Film.java — solution proposée

```java
package net.lecnam.ecue2.tp7.film;

import java.time.Year;

// implements Comparable<Film> : définit l'ORDRE NATUREL des films
public class Film implements Comparable<Film> {

    private final String titre;
    private final String realisateur;
    private final Year anneeSortie;          // java.time.Year : une année

    public Film(String titre, String realisateur, int annee) {
        this.titre = titre;
        this.realisateur = realisateur;
        this.anneeSortie = Year.of(annee);   // conversion int -> Year
    }

    // Getters uniquement (pas de setters)
    public String getTitre() { return titre; }
    public String getRealisateur() { return realisateur; }
    public Year getAnneeSortie() { return anneeSortie; }

    @Override
    public String toString() {
        return titre + ", " + realisateur + ", " + anneeSortie;   // titre, realisateur, aaaa
    }

    // Exercice 4 : comparaison sur l'année uniquement
    //   return this.anneeSortie.compareTo(f.anneeSortie);
    // Exercice 7 : année PUIS titre (sinon le TreeSet élimine les films de même année)
    @Override
    public int compareTo(Film f) {
        int cmp = this.anneeSortie.compareTo(f.anneeSortie);   // Year est Comparable
        if (cmp != 0) {
            return cmp;                       // années différentes : elles décident
        }
        return this.titre.compareTo(f.titre); // même année : ordre alphabétique du titre
    }
}
```

Execute.exercice4() — solution proposée

```java
private static void exercice4() {
    // TreeSet : collection NATIVEMENT triée (utilise compareTo de Film)
    Set<Film> films = new TreeSet<>();
    films.add(new Film("Le Parrain", "Francis Ford Coppola", 1972));
    films.add(new Film("Pulp Fiction", "Quentin Tarantino", 1994));
    films.add(new Film("Le Voyage de Chihiro", "Hayao Miyazaki", 2001));
    films.add(new Film("Les Évadés", "Frank Darabont", 1994));
    films.add(new Film("Parasite", "Bong Joon-ho", 2019));

    for (Film f : films) System.out.println(f);     // trié par année croissante

    // ArrayList construite à partir de la collection (copie des éléments)
    List<Film> liste = new ArrayList<>(films);
    Collections.reverse(liste);                     // inverse l'ordre : années décroissantes
    System.out.println("----------------");
    for (Film f : liste) System.out.println(f);
}
```

Exercices 5 à 7 — FilmService (import CSV, recherche, index par année)

Format de `films.csv` (1406 lignes, UTF-8) : `annee;titre;realisateur`, par exemple `1990;Camille Claudel;Nuytten, Bruno`. Le séparateur est le **point-virgule** car le nom du réalisateur contient une virgule.

film/FilmService.java — solution proposée

```
package net.lecnam.ecue2.tp7.film;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Year;
import java.util.*;

public class FilmService {

    private static Set<Film> filmSet;                    // tous les films, triés
    private static Map<Year, Set<Film>> indexParAnnee;   // exercice 7 : année -> films

    public FilmService() throws IOException {
        filmSet = new TreeSet<>();
        indexParAnnee = new HashMap<>();
        importFilm();                                    // les films sont chargés à la création
    }

    // Exercice 5 : lecture du CSV et création des objets Film
    private void importFilm() throws IOException {
        List<String> lignes = Files.readAllLines(Path.of("films.csv"), StandardCharsets.UTF_8);
        for (String ligne : lignes) {
            if (ligne.isBlank()) continue;               // ignore les lignes vides
            String[] champs = ligne.split(";");          // [annee, titre, realisateur]
            int annee = Integer.parseInt(champs[0].trim());
            Film film = new Film(champs[1].trim(), champs[2].trim(), annee);
            filmSet.add(film);

            // Exercice 7 : ajout dans l'index.
            // Si l'année n'est pas encore une clé, on crée un nouveau sous-ensemble.
            Set<Film> filmsDeLAnnee = indexParAnnee.get(film.getAnneeSortie());
            if (filmsDeLAnnee == null) {
                filmsDeLAnnee = new TreeSet<>();
                indexParAnnee.put(film.getAnneeSortie(), filmsDeLAnnee);
            }
            filmsDeLAnnee.add(film);   // on modifie l'ensemble DÉJÀ rangé dans la map
            // Version courte équivalente :
            // indexParAnnee.computeIfAbsent(film.getAnneeSortie(), k -> new TreeSet<>()).add(film);
        }
    }

    public void printFilms() {
        for (Film f : filmSet) System.out.println(f);
    }

    // Exercice 6 : films dont le titre CONTIENT la chaîne (insensible à la casse)
    public void searchFilm(String search) {
        String s = search.toLowerCase();
        for (Film f : filmSet) {
            if (f.getTitre().toLowerCase().contains(s)) {
                System.out.println(f);
            }
        }
    }

    // Exercice 7 : accès direct par la clé, sans parcourir les 1406 films
    public void searchFilmByAnnee(int search) {
        Set<Film> films = indexParAnnee.getOrDefault(Year.of(search), Collections.emptySet());
        System.out.println(films.size() + " film(s) en " + search);   // 1985 -> 39
        for (Film f : films) System.out.println(f);
    }
```

film/FilmService.java — solution proposée

```
}

// Dans Execute :
// exercice5() : try { new FilmService().printFilms(); } catch (IOException e) { ... }
// exercice6() : service.searchFilm("man");
// exercice7() : service.searchFilmByAnnee(1985);
```

À retenir : L'index `Map<Year, Set<Film>>` illustre l'intérêt d'une `HashMap` : retrouver les films d'une année se fait en **une opération** (`get`) au lieu de parcourir toute la collection.

Projet « untitled » (modèle IntelliJ)

untitled/src/Main.java

```java
// Fichier modèle généré par IntelliJ (syntaxe Java 25 : main sans classe)
void main() {
    IO.println(String.format("Hello and welcome!"));
    for (int i = 1; i <= 5; i++) {
        IO.println("i = " + i);
    }
}
```

4. Résumé express

- `List` = ordre + doublons ; `Set` = sans doublons ; `Map` = clé → valeur.
- `ArrayList`, `HashSet`/`TreeSet` (trié), `HashMap`/`TreeMap` (clés triées).
- Déclarer avec l'interface : `List<String> l = new ArrayList<>();`.
- Parcours : for (indices), for-each, `Iterator` (seul à permettre `remove` pendant le parcours).
- `Collections.sort`, `Collections.reverse` ; `new TreeSet<>(liste)` trie et dédoublonne.
- `Comparable<T>` + `compareTo` : négatif / 0 / positif ; dans un TreeSet, 0 = doublon !
- Index : `map.computeIfAbsent(cle, k -> new TreeSet<>()).add(valeur)`.